



제가 만든 환경이
팀의 발판이 되는 순간,
비로소 인프라의 의미가 완성됩니다.

DevOps/Infra

규정을 기술로 구현하기 위해
안정성, 보안, 컴플라이언스, 자동화에 우선순위를 둡니다.

Contact

Phone 010-5321-1503
Email yo12399@naver.com
GitHub <https://github.com/Joyeongbinnn>

Best Skills



Docker



멀티 스테이지 빌드를 활용하여 경량 Docker 이미지 제작이 가능합니다.
네트워크·볼륨 설정, 컨테이너 간 통신을 구성할 수 있습니다.
Docker-Compose.yaml, Dockerfile 을 작성할 수 있습니다.



Jenkins



Declarative Pipeline/Jenkinsfile 을 작성할 수 있습니다..
Kaniko 기반 No-Docker Build, Trivy 취약점 검사하고
ECR Push 자동화를 할 수 있습니다.
GitOps 에 맞추어 CI/CD 직접 설계 및 적용해 본 경험이 있습니다.



AWS EC2



AWS S3



인스턴스를 활용해 CI 노드 + K3s Cluster 환경 구성
서브넷, 라우팅, 보안그룹 구성 후 CI ↔ Cluster 간 통신 구조 설계
최소 권한 원칙 기반 ECR·S3 접근 Role/Policy 설정을 할 수 있습니다.



Linux



네트워크, 파일 시스템 관리, 사용자 권한 설정을 할 수 있습니다.
가장 익숙한 환경은 Ubuntu 입니다.



Java



객체지향 설계, 예외 처리, REST API 구조에 대한 이해.
java 와 자료구조를 이용해 알고리즘 문제를 해결할 수 있습니다.

Skills



Spring Boot



JWT+Spring Security 기반 인증 및 권한 처리 구현 경험이 있습니다.
REST API, CRUD, MVC 기반 서비스 개발 가능.
AWS S3 Presigned URL 업로드 기능 및 서버-프론트 연동 경험이 있습니다.



GitOps(ArgoCD)



Kubernetes 환경에 Git 기반 매니페스트 적용.
ECR 이미지 변화를 인식하도록 kustomization 으로 자동화 했습니다.



Mysql



Join/Sub Query 를 능숙하게 사용합니다.
MyBatis 기반 Query 튜닝 및 페이징을 처리할 수 있습니다.



Git



Branch 전략을 세워 프로젝트를 진행했습니다.
CI/CD 와 연동하여 자동화 파이프라인을 구성할 수 있습니다.

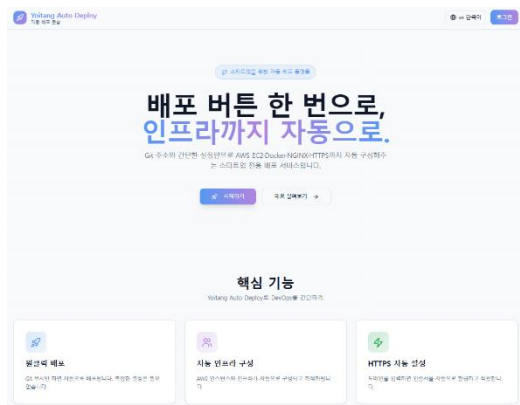


Kubernetes (K3s Multi Node)



EC2 기반 4 대 노드 구성 (CI Node 1 + Master 1 + Worker 2).
Ingress, Service, Deployment, Secret, TLS 인증 설정을 할 수 있습니다.

Projects(1/4) (Latest)



Yoitang 2025.11.14 ~ 2025.11.23

원클릭 자동배포 플랫폼

2025softbank hackathon 1round

<https://github.com/Joyeongbinnn/2025softbank-hackathon-preliminary->



2025 SoftBank Global Hackathon Korea 최우수상

Yoitang Auto Deploy - 원클릭 자동 배포 플랫폼

심사위원 평가

KIM@SB: "유저 측 목선에서의 편리성 고려, 용장성의 관점, 코스트에 대해서도 생각하고 있는 점이 좋았다."

Makoto Shimazu (Progate): "높은 기술 레벨로 정리하고 있어 훌륭하다. 세세한 기능까지 포함하여 만들어져서 매우 훌륭하다."

핵심 기술

k3s Multi Control-Plane

Jenkins + Kaniko

AWS ECR

HTTPS 자동화

Private Repository 지원

보안 스캔 (Trivy)

인프라 개발환경

서버

AWS EC2 (CI Node : t3.large, Cluster : t3.large)

기술

Docker, Jenkins, Kaniko, Trivy, Kubernetes(K3s), Ingress, cert-manager, Docker Registry(ECR)

소스 관리

GitHub (Public/Private Repo, PAT 인증 지원)

IDE

nano, VSCode, Shell Script

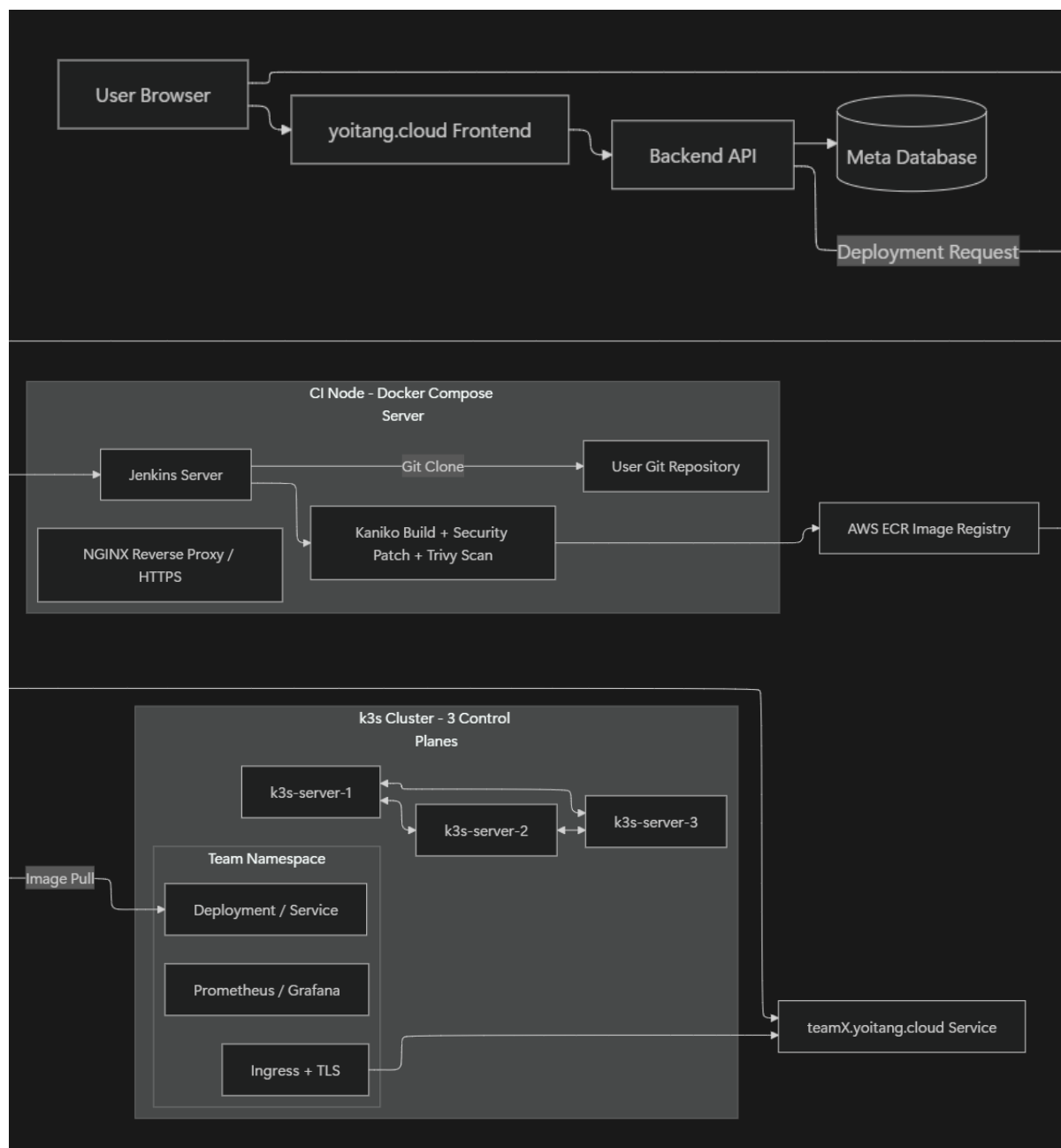
Projects(1/4) (Latest)

담당 역할

기획 / DevOps / 배포 자동화 파이프라인 구축 / k3s 클러스터 구축 / 발표

젠킨스 자동배포 파이프라인

Git -> Jenkins(kaniko) 빌드 -> ECR push -> K3s(ECR pull) -> Https 배포

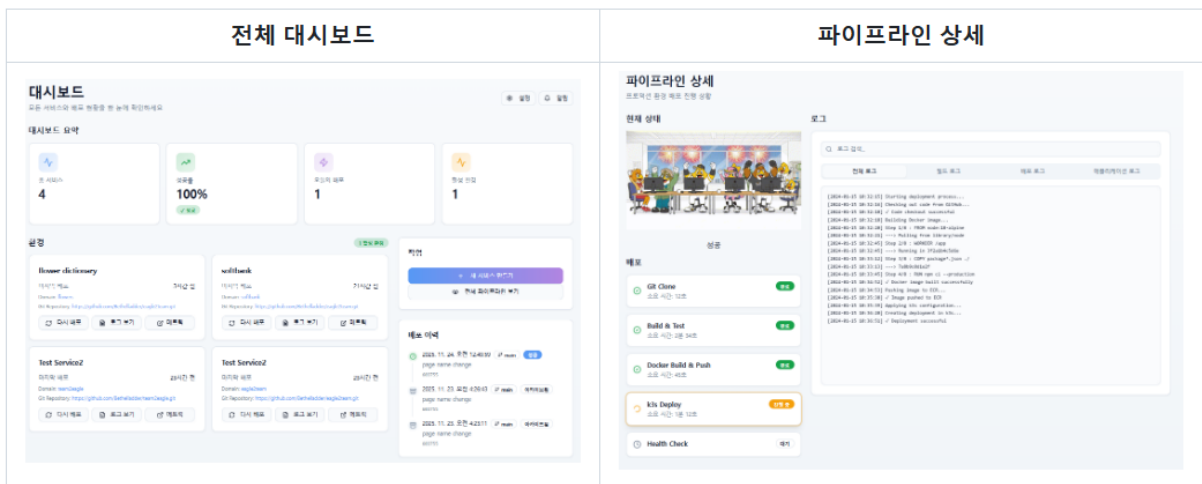


Projects(1/4) (Latest)

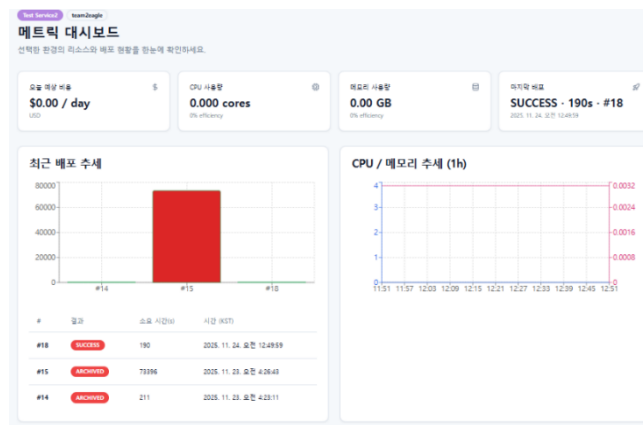
배포된 팀/서비스 관리

쿠버네티스 클러스터를 선택한 이유는 다음과 같습니다.

1. 팀/서비스별 Namespace 를 분리하여 격리실행환경을 제공하기 위해서
2. 사용자가 어떤 프레임워크를 쓰든 동일한 배포방식 제공가능
3. 사용자가 많아질수록 수평확장이 가능하게 하기 위해서
4. pod 재시작, 롤링업데이트 등 Auto Healing 을 위해서

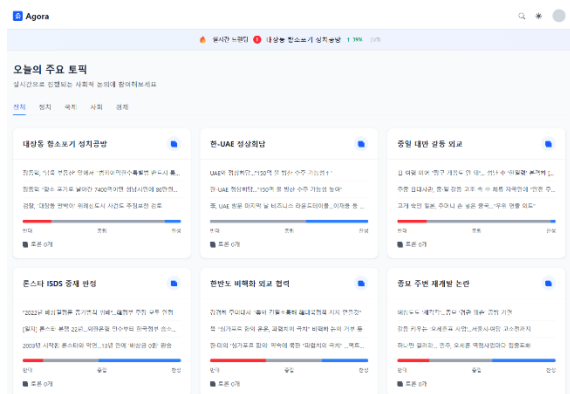


다음은 grafana + Prometheus + kubecost 를 이용한 메트릭 대시보드 입니다.



사용자(namespace)별 cpu 사용량, 배포 추세, 그에따라 부과되는 비용을 볼 수 있습니다.

Projects(2/4)



Agora

2025.10.21 ~ 2025.11.21

찬반, 정파, 시각별 뉴스와 댓글을 AI 로
분류하여 여론의 현황을 제시하는 플랫폼

SSAFY 3 차 프로젝트

5 인(Front 1, back 2, AI 2, Infra 1)

담당 역할

Infra 구성

ec2 1 대에는 CI 노드를, ec2 3 대에 kubernetes 클러스터
설계 및 구축,

CI/CD 파이프라인 구축

Jenkins(kaniko + trivy) -> ECR -> ArgoCD Auto
Sync 로 이어지는 GitOps 파이프라인 설계/구현

배포 · 네트워크 구조 설계

Edge Nginx -> K3s Ingress -> 서비스로 이어지는
단일도메인 라우팅 구조 설계

보안/컴플라이언스

ECR pull Secret CronJob, 최소권한 SG/IAM 정책 설계

인프라 개발환경

서버

AWS EC2, ECR,

기술

ArgoCD, Docker, K3S, Jenkins, Docker-compose

소스

GitLab

IDE

nano, VSCode

Projects(2/4)

문제 해결 사례

1. ECR 토큰 만료로 인한 이미지 pull 실패

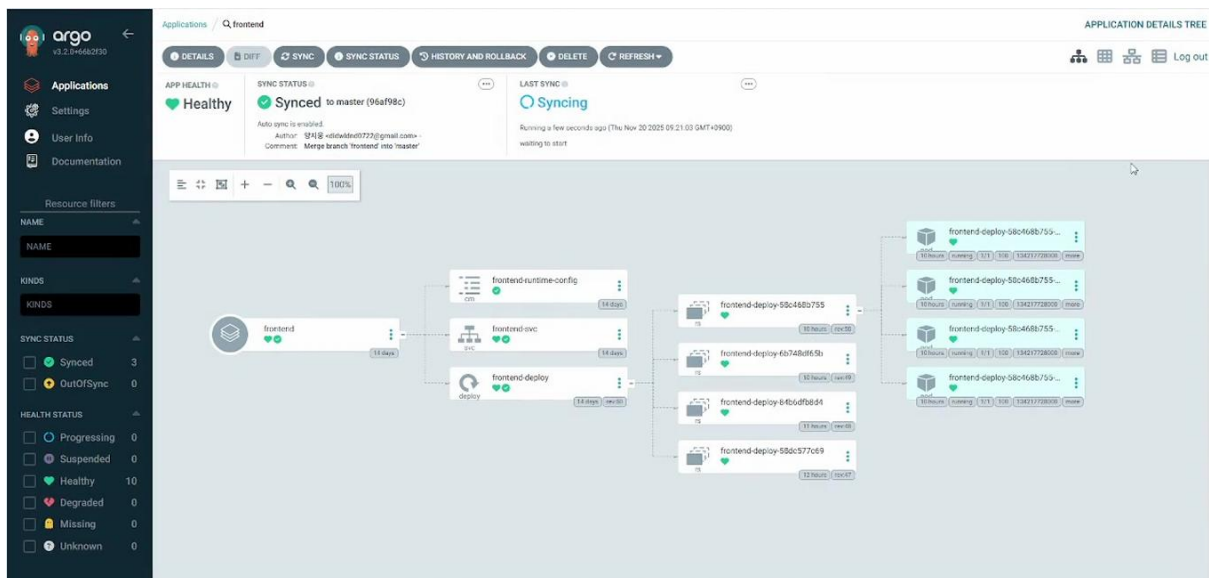
k3s 는 AWS IAM 인증을 네이티브로 지원하지 않아 **ECR 토큰이 12 시간마다 만료**되면서 pod 재시작시 이미지 pull 이 실패하는 배포 장애가 발생했습니다.

대응 및 해결:

- A. Kubernetes CronJob 을 활용한 ECR 토큰 재발급 시스템 구축
- B. Secret 자동 업데이트 로직 구현
- C. ArgoCD Sync 시점과 충돌 방지위해 시간차 배치 스케줄링

2. ArgoCD 의 신규 이미지 감지 오류

ECR 에 새로운 이미지가 Push 되었음에도 **ArgoCD 가 변경사항을 감지하지 못해** 자동배포가 이루어지지 않았습니다.



해결 방법 :

- A. Kustomize 기반 이미지 태그 자동 반영 정책 추가
- B. GitOps 선언형 배포 방식으로 완전히 통일

Projects(2/4)

3. CI 노드 Docker 중복 설치로 인한 서비스 장애

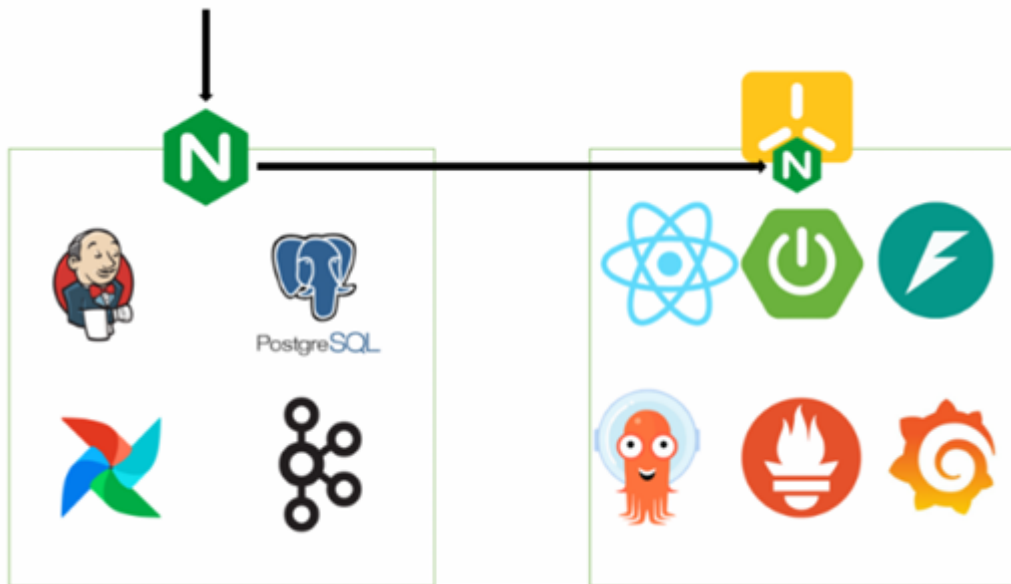
팀원이 Docker Compose 로 실행중인 컨테이너 환경 위에 **별도의 Docker 엔진을 설치**하면서 충돌이 발생, Jenkins 와 PostgreSQL 등 핵심 서비스 다운

대응 및 해결:

- A. 포트스캔 후 유령 프로세스 제거
- B. Docker 환경 복원 및 볼륨 초기화
- C. 인프라 변경 시 사전 승인 프로세스 도입

4. 단일 도메인/서브패스 에서 TLS/라우팅 꼬임 문제

Grafana, Next.js, 정적 리소스, Jenkins 를 모두 k13b101.p.ssafy.io 에 붙이면서 **서브패스 · 헤더 · root_url 설정이 맞지 않아** 502, 리소스 로드 실패 발생



대응 및 해결:

- A. Edge Nginx 에서 location 별 proxy 설정과 헤더 Prefix 통일
- B. Grafana root_url, server_from_sub_path 조정으로 해결

Projects(3/4)



KDUCK

Kduck

2025.09.01 ~ 2025.09.29

K-POP NFT 마켓플레이스

SSAFY 2 차 프로젝트

엔터사 발행 NFT 팩을 구매하고 팬 간의
자유로운 거래를 통해 자산을 교환

6 인(Front 2, back 3, Infra 1)

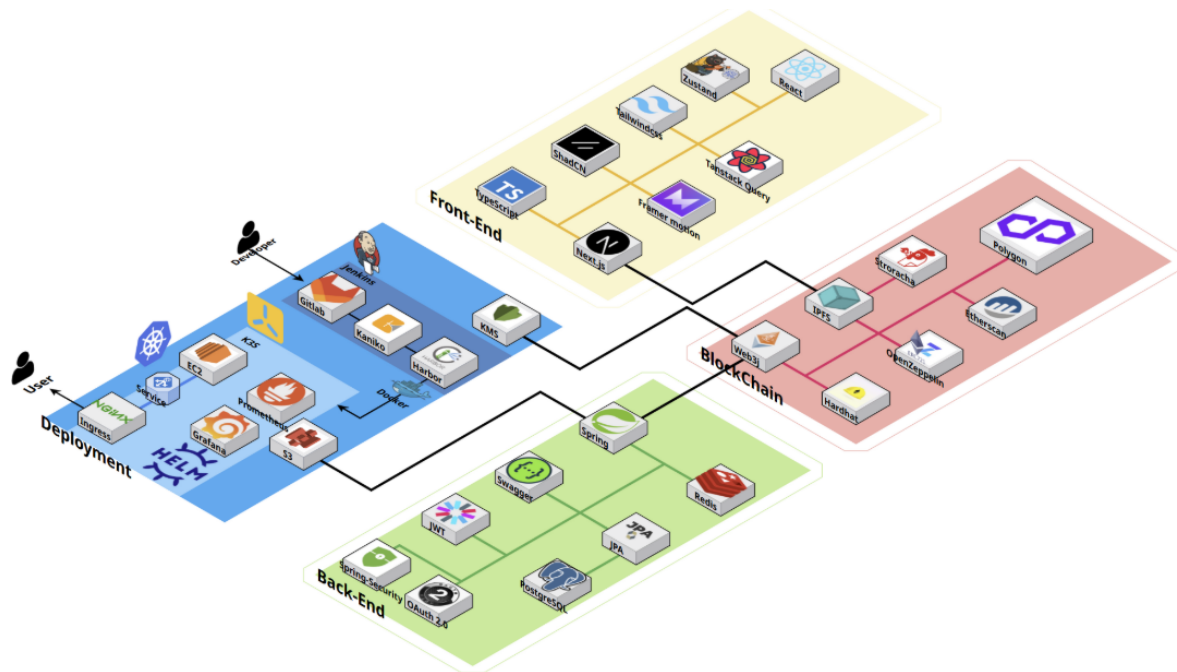
담당 역할

기획	NFT 거래 모델, 민팅 구조, 거래수수료 정책 설계
Infra 구성	AWS EC2 단일 노드에 K3s 클러스터 구축, IPFS 연동
CI/CD 파이프라인 구축	GitLab Webhoob -> Jenkins -> Harbor -> K3s
보안/컴플라이언스	NFT 민팅 key 암호화(KMS), Secret 분리, Private Registry 이미지 스캔

인프라 개발 환경

서버	AWS EC2, IPFS Gateway(Web3.Storage), KMS
사용 기술	Docker, Kubernetes(K3s), Jenkins, Kaniko, Harbor
소스 관리	GitLab (Webhook 기반 자동화)
IDE	nano, VSCode

Projects(3/4)



문제 해결 사례

1. 이미지 빌드 실패

CI 파이프라인에서 Docker 이미지를 직접 빌드하려 했으나, 파이프라인이 실행되는 Pod 내부에는 **Docker 데몬이 없어 빌드가 반복적으로 실패.**

```
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE
```

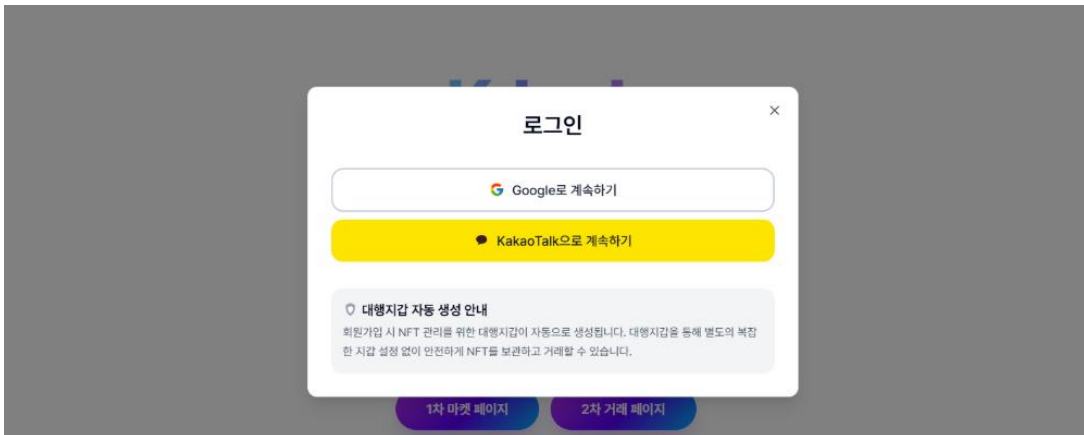
대응 및 해결:

- Docker-in-Docker(DinD) 방식 검토 후 루트 권한 · 소켓 노출 등 보안위험 확인
- 데몬 없이 이미지 빌드 가능한 **kaniko** 로 전환
- Kaniko 기반 CI 빌드 및 레지스트리 push 자동화 구축

Projects(3/4)

2. OAuth2 로그인 세션 불안정

로그인정상 로그인 상태임에도 화면 이동 · API 요청 시 간헐적으로 “다시 로그인하세요” 메시지가 발생.



대응 및 해결:

- A. Ingress의 라운드로빈 분산으로 요청이 매번 다른 파드로 전달되는 현상 확인
- B. 세션 쿠키가 파드 간 공유되지 않아 인증 상태가 끊기는 구조 분석
- C. Ingress에 Cookie 기반 세션 어피니티(Stricky Session) 적용하여 동일 파드로 라우팅

```
nginx.ingress.kubernetes.io/affinity: "cookie"
```

3. Ingress 통합 라우팅으로 인해 /api 응답이 실패하는 문제

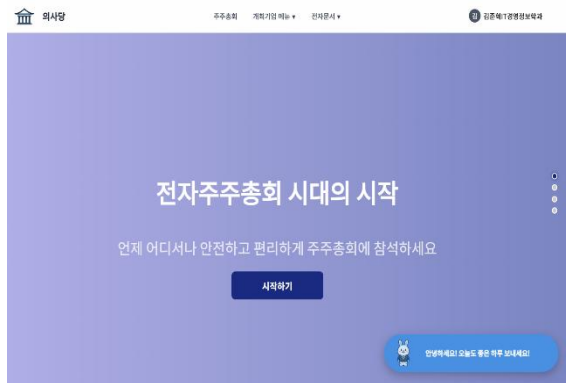
화면은 정상 출력되지만 API 호출에서 **404 또는 502 응답**이 간헐적으로 발생.

대응 및 해결:

- A. **"/" 라우팅이 "/api" 경로보다 우선 매칭**되어 API 요청이 Next.js로 잘못 전달되는 문제 확인
- B. 라우팅 우선순위 불명확으로 인한 Ingress 규칙 충돌 분석
- C. **"/api" 경로를 별도 Ingress 규칙으로 분리**하여 우선순위 명확화 및 API 트래픽 독립 처리

```
! 99-ingress-api.yaml
! 99-ingress.yaml
```

Projects(4/4)



의사당

2025.07.07 ~ 2025.08.15

전자주주총회 플랫폼 - 의사당

상법 개정에 따른 전자주주총회 의무화를
대비한 실시간/법적 증빙 중심 플랫폼

SSAFY 1 차 프로젝트

6 인(Front 3, back 2, Infra 1)

담당 역할

PM/기획	개정 상법 요구사항 -> 기술 요구사항으로 변환 증빙 중심 시스템 요구 정의(로그 무결성, 인증인가)
Infra 설계/구성	EC2 단일 노드에 Docker Compose, WebRTC 서버
CI/CD 파이프라인 구축	GitLab Webhook -> Jenkins -> docker compose
보안/컴플라이언스	상법, 개인정보보호법, 예탁결제원 가이드라인

인프라 개발 환경

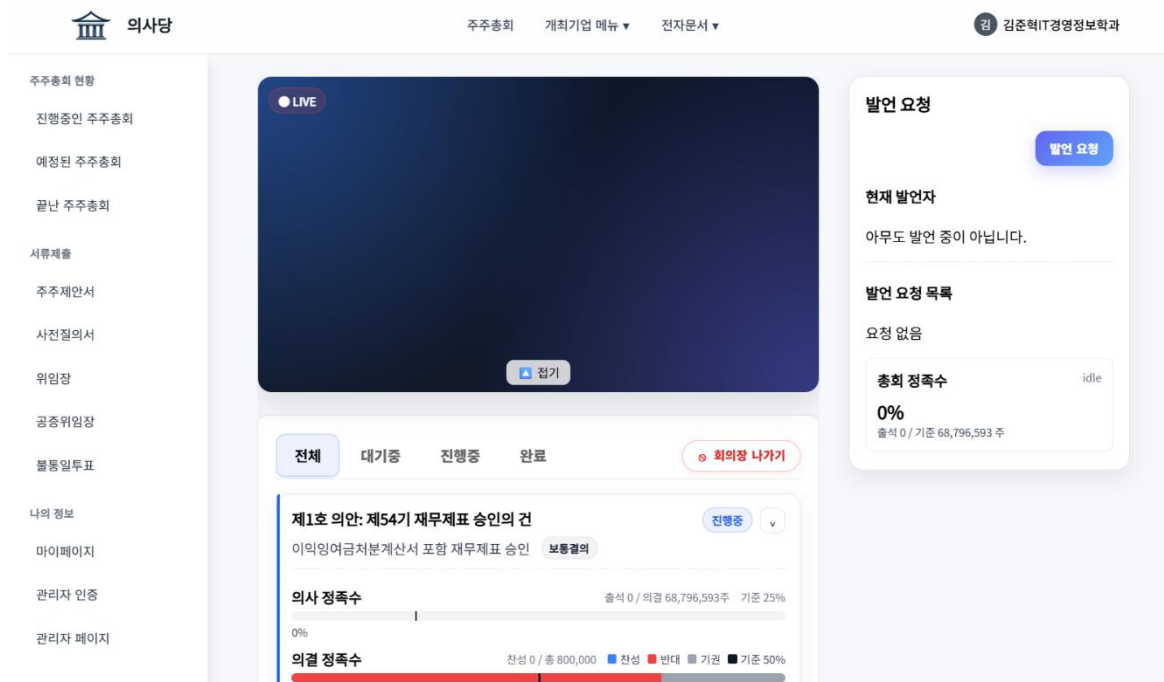
서버	AWS EC2, S3, OpenVidu 3.3, LiveKit
사용 기술	Docker-compose, Jenkins, Nginx
소스 관리	GitLab (Webhook 기반 자동화)
IDE	nano, VSCode

Projects(4/4)

문제 해결 사례

1. OpenVidu 3.3 학습 및 실시간 통신 구조 적용

최신 버전(Openvidu 3.3) 기반으로 서버를 구축해야 했으나, [문서 · 예제가 부족하고 WebRTC 및 WebSocket 규약이 생소](#)해 시행착오가 반복됨



대응 및 해결:

- [공식 예제](#)를 직접 실행해 Docker Compose 기반으로 통합하며 네트워크 구조 및 시그널링 순서 분석
- WebRTC 연결흐름과 WebSocket 제어 규약을 단계별로 확인
- 기반 구조를 커스터마이징하여 [실시간 회의에 전자투표기능](#)까지 확장 구현

2. 환경변수 주입 시점 오류(Jenkins + Docker Compose)

Spring Boot 설정과 Docker Compose 실행 환경이 섞여 .env 파일을 여러 위치에 중복 배치해야 하는 비효율이 발생.

```
backend:
  build: ./backend
  image: deploy-backend
  container_name: backend

  ports:
    - "8080:8080"

  env_file:
    - .env

  environment:
    - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/${POSTGRES_DB}
    - SPRING_DATASOURCE_USERNAME=uisadang
    - SPRING_DATASOURCE_PASSWORD=${POSTGRES_PASSWORD}
```

대응 및 해결:

- A. Docker Compose env_file 적용 시점과 Spring Boot 의 환경변수 적용 우선순위 혼동 문제 확인
- B. JVM 실행 시 application.properties 매핑 구조 및 환경변수 우선순위 차이 분석
- C. 환경변수를 단일 .env 로 통합하고 Compose 를 진입점으로 설정해 컨테이너에서만 로드 후 JVM 으로 전달되도록 구조 정리

3. MongoDB + S3 기반 로그 무결성 보장

인증, 회의, 투표 기록이 내부 MongoDB 에만 저장될 경우, 조작 시 추적이 어렵고 감사 과정에서 법적 증빙 자료로 인정받기 어려움

대응 및 해결:

- A. 단일 DB 저장 시 로그 변경이 원본 증빙 검증을 불가능하게 만드는 구조적 위험 확인
- B. 로그에 거래 해시, 타임스탬프, IP 정보를 포함하면 무결성 검증 가능하다는 점 분석
- C. 로그 저장방식을 MongoDB + AWS S3 이중저장으로 분리 설계하여 로그의 무결성과 추적성 및 법적 증빙 가능성 강화
 - i. SHA-256 해시 형태로 S3 에 보존
 - ii. 빠른 검색 · 조회는 Mongo 로 처리

```
{
  "$jsonSchema": {
    "bsonType": "object",
    "required": [
      "_id",
      "_class",
      "createdAt",
      "eventTimeline",
      "finishedAt",
      "participants",
      "roomName",
      "status"
    ],
    "properties": {
      "_id": {
        "bsonType": "objectId"
      },
      "_class": {
        "bsonType": "string"
      },
      "createdAt": {
        "bsonType": "long"
      },
      "eventTimeline": {
        "bsonType": "array",
        "items": {
          "bsonType": "object"
        }
      }
    }
  }
}
```